AmanAI Lab — Agentic AI Interview Prep Series Day 1 Complete Notes —Multi-Agents System Concepts & Real Interview Questions
Series — Agentic AI Interview Prep | Day 3 of 5 | @AmanAI_lab

Day 3 Agentic AI Interview (Multi-Agent Systems)

Q1. At what point do you say *"this needs to be multi-agent"* and what's your first split? *(Tests your system design instinct)?*

◇ 1. Start with the Core Principle (Very Important in Interview)

I **don't start with multi-agent by default**.

I first try to solve the problem with a **single agent + tools + structured prompting**.
I move to multi-agent only when **complexity, specialization, or control breaks down**.

◇ 2. When Do I Decide Multi-Agent is Needed?

I switch to multi-agent when I observe **one or more of these signals:**

☑ **(A) Task Decomposition Becomes Natural**

- Problem can be clearly broken into **independent steps**
- Example:
  - Input understanding
  - Retrieval
  - Reasoning
  - Output formatting

👉 If steps have **different objectives**, single agent struggles.

☑ **(B) Prompt Becomes Too Large / Fragile**

- One prompt trying to do everything:
  - reasoning + tool use + formatting + validation

❌ Leads to:

- hallucinations
- instruction conflicts
- inconsistent outputs

👉 Signal: "Prompt engineering is becoming unmanageable"

☑ (C) Need for Specialized Expertise

Different parts need **different capabilities**:

- Retriever → search optimized
- Analyst → reasoning heavy
- Formatter → structured output

👉 One model doing all = suboptimal

☑ (D) Iteration / Feedback Loops Needed

- Example:
  - Generate → Critique → Improve

👉 This is naturally **multi-agent (generator + critic)**

☑ (E) Tool Overload

- Too many tools:
  - DB
  - APIs
  - calculators
  - vector search

👉 Better to assign tools to **specific agents**

☑ (F) Need for Reliability / Guardrails

- Want:
  - validation
  - fact-checking
  - structured enforcement

👉 Add a **validator agent**

◇ 3. My First Split (Most Important Part)

When I decide to go multi-agent, I **don't overcomplicate**.

I start with a **minimal, high-impact split:**

★ Default First Split: Planner + Executor

1. Planner Agent

- Breaks problem into steps
- Decides:
  - what to do
  - which tool to use
  - order of execution

👉 Output: structured plan

---

2. Executor Agent

- Executes each step
- Calls tools
- Generates outputs

👉 Focus = action, not planning

Why this split works:

- Separates **thinking vs doing**
- Reduces hallucination
- Improves control and traceability

---

4. If System Grows → Next Splits

Once system evolves, I extend:

Add Retriever Agent

- Handles RAG separately
- Improves search relevance

Add Critic / Validator Agent

- Checks:
  - correctness
  - format
  - hallucination

Add Memory / Context Agent

- Manages:
  - conversation history

- long-term memory

🔹 5. Example (Interview-Level)

🎯 Use Case: Financial Document Analysis

❌ Single Agent Problem:

- Reads doc
- extracts data
- validates
- summarizes

👉 Too much responsibility → errors

☑ Multi-Agent Design:

1. **Planner**
   - Identify sections (income, assets, risks)
2. **Retriever**
   - Fetch relevant document chunks
3. **Analyzer**
   - Extract financial insights
4. **Validator**
   - Cross-check numbers
5. **Formatter**
   - Generate final structured report

🔹 6. Key Interview Line (Must Say)

"I move to multi-agent not for complexity, but for **separation of concerns, reliability, and scalability**."

🔹 7. Common Mistake (Call This Out)

❌ Over-engineering:

- Creating 5–6 agents too early

✔ Better:

- Start with **2 agents**
- Expand only when needed

🔹 Final Crisp Answer (If Interviewer Interrupts)

I start with a single agent and move to multi-agent when tasks become decomposable, prompts get complex, or specialization is needed.
My first split is usually **Planner vs Executor**, separating decision-making from execution.
Then I incrementally add agents like retriever or validator based on system needs.

Q2. Walk me through exactly what happens when an orchestrator sends a task to a sub-agent — who holds state, who decides if the result is good enough? *(Tests depth on the core pattern)*

◇ 1. Start with the Big Picture

When an orchestrator sends a task to a sub-agent, the flow involves task packaging → execution → evaluation → feedback → state update.

There are 3 key responsibilities:

- **Orchestrator → control + decision-making**
- **Sub-agent → execution**
- **State store → memory + traceability**

◇ 2. Step-by-Step Flow (This is the core answer)

🟣 Step 1: Orchestrator Creates Task

The orchestrator:

- **Breaks the plan into a task**
- **Defines:**
  - **objective**
  - **constraints**
  - **expected output format**
  - **success criteria**

👉 Example:

```
Task:
- Extract revenue from document
- Return JSON: { "revenue": number }
- Confidence > 90%
```

🟣 Step 2: State is Read (Before Execution)

The orchestrator pulls from state store:

- **previous steps**
- **conversation history**
- **retrieved documents**

- intermediate outputs

👉 **Important:**

Orchestrator is the primary reader of global state

🟣 **Step 3: Task Sent to Sub-Agent**

Sub-agent receives:

- task instructions
- relevant context (not full state, only scoped context)
- tool access (if needed)

👉 **Key idea:**

Sub-agent is stateless or minimally stateful

🟣 **Step 4: Sub-Agent Executes**

Sub-agent:

- reasons over input
- may call tools (RAG, APIs, DB)
- produces output

👉 **Output includes:**

- result
- optional metadata (confidence, reasoning, sources)

🟣 **Step 5: Result Written to State**

The result is stored in:

- shared memory / state store

👉 **Now system has:**

- traceability
- reproducibility

🟣 **Step 6: Who Decides if Result is Good Enough?**

👉 **This is the most important part**

☑ **Primary Decision Maker: Orchestrator**

The orchestrator:

- checks:
  - format correctness
  - completeness
  - confidence
  - alignment with task

☑ **Optional: Validator / Critic Agent**

In more advanced systems:

A separate agent:

- reviews output
- detects:
  - hallucination
  - logical errors
  - missing fields

🟣 **Step 7: Feedback Loop (If Not Good Enough)**

If result fails:

Orchestrator can:

1. Retry same agent with better prompt
2. Call different agent
3. Add more context (e.g., more retrieval)
4. Escalate to human (in production systems)

🟣 **Step 8: State Update + Next Step**

Once accepted:

- state is updated
- orchestrator moves to next task

🔸 **3. Who Holds State? (Very Important)**

☑ **Centralized State Store (Best Practice)**

State is NOT held inside agents.

Instead:

- **stored in:**
  - **memory DB**
  - **vector DB**
  - **workflow state (e.g., LangGraph)**

◇ Roles:

Component	Responsibility
Orchestrator	Reads + writes global state
Sub-agent	Uses scoped context only
State Store	Source of truth

💡 Key Line for Interview:

"Agents are kept mostly stateless; the orchestrator and external memory manage state to ensure consistency and debuggability."

◇ 4. Who Decides Quality?

☑ Hierarchy:

1. **Orchestrator (always)**
2. **Validator/Critic (optional but recommended)**
3. **Human-in-the-loop (for high-stakes systems)**

💡 Another Strong Line:

"Quality decisions are centralized to avoid inconsistent judgments across agents."

◇ 5. Example (Concrete)

🎯 Use Case: Commodity Insight Extraction

Flow:

1. **Orchestrator:**

- Task: Extract aluminum price trend

2. Sub-agent:
   - retrieves articles
   - analyzes trend
3. Output:

```
Trend: Increasing
Confidence: 0.82
```

4. Orchestrator checks:

- Confidence < 0.9 ❌

5. Action:

- triggers re-run with more documents

---

6. Common Mistake (Say This)

❌ **Letting sub-agents decide:**

- "I think this is correct"

✔ **Instead:**

Always validate externally

---

7. Final Crisp Answer (For Interview)

When the orchestrator sends a task, it packages the objective, constraints, and context from a centralized state store. The sub-agent executes with scoped context and returns a result. The result is written back to the shared state. The orchestrator then evaluates whether it meets success criteria, optionally using a validator agent. If not, it triggers retries or adjustments. State is centrally managed, and quality decisions are made by the orchestrator to ensure consistency.

Q3. Explain MCP to me like I'm a backend engineer who's never touched agents. Then tell me where you'd use it and where you wouldn't. *(MCP is red-hot right now — every top company asks this)*

1. What is MCP (in one line)

MCP (Model Context Protocol) is a **standard way for an AI model to talk to external systems (tools, APIs, data sources) in a structured, discoverable, and safe way.**

◇ 2. Translate MCP into Backend Language

Think of MCP like this:

If REST is **API-to-API communication**, MCP is **LLM-to-API communication with structure + schema + discovery**.

Backend Analogy:

Backend Concept	MCP Equivalent
REST API	MCP Server
Endpoint	Tool
OpenAPI spec	Tool schema
API Gateway	MCP layer
Client	LLM / Agent

◇ 3. Without MCP vs With MCP

❌ Without MCP (what most people do initially)

- Hardcode tools in prompts:

```
"You can call function get_weather(city)"
```

Problems:

- brittle
- not scalable
- poor tool discovery
- tight coupling

☑ With MCP

- Tools are exposed via a **standard protocol**
- Model can:
  - discover tools
  - understand schema
  - call them reliably

👉 It becomes **plug-and-play for tools**

◇ 4. How MCP Actually Works (Step-by-Step Flow)

🟣 Step 1: MCP Server exposes tools

Example tools:

- `get_stock_price`
- `query_database`
- `search_documents`

Each tool has:

- name
- input schema (JSON)
- description

🟣 Step 2: Model connects to MCP server

Instead of hardcoding tools:

- model queries:
  - "What tools are available?"

🟣 Step 3: Tool Discovery

Model receives:

```
{
  "name": "get_stock_price",
  "input": { "ticker": "string" }
}
```

👉 This is like **OpenAPI for LLMs**

🟣 Step 4: Model Decides to Call Tool

User asks:

"What is the current price of Apple stock?"

Model:

- identifies tool
- generates structured call

🟣 Step 5: MCP Executes Tool

- MCP server calls actual backend API
- returns result

🟣 Step 6: Model Uses Result

- integrates into final response

🔷 5. Real Example (Interview-Ready)

🎯 Use Case: Financial Assistant

User:

"Give me latest aluminum price and summarize trend"

❌ Without MCP:

- You manually:
  - wire APIs
  - write function calls
  - manage schema in prompt

☑ With MCP:

Tools exposed:

- `get_commodity_price`
- `get_market_news`

Flow:

1. Model discovers tools
2. Calls:

```
{ "tool": "get_commodity_price", "commodity": "aluminum" }
```

3. MCP hits backend (your API)
4. Returns:

```
{ "price": 2450 }
```

5. Model combines with news → gives insight

◇ 6. Where I Would Use MCP ☑

☑ (A) Many Tools / APIs

- enterprise systems
- internal APIs
- microservices

👉 MCP gives **standardization**

☑ (B) Dynamic Tooling

- tools change frequently
- new APIs added

👉 No need to rewrite prompts

☑ (C) Multi-Agent Systems

- multiple agents need shared tools

👉 MCP acts like **central tool registry**

☑ (D) Strong Backend Integration

- DB queries
- internal services
- analytics pipelines

👉 Clean separation of concerns

☑ (E) Security & Governance

- control:
  - which tools model can access
  - logging
  - auditing

◇ 7. Where I Would NOT Use MCP ❌

❌ **(A) Simple Use Case**

- 1–2 tools only

👉 MCP = overkill

❌ **(B) Static Workflows**

- fixed pipeline
- no dynamic decisions

👉 Use normal orchestration

❌ **(C) Low Latency Critical Systems**

- MCP adds:
  - discovery step
  - abstraction overhead

👉 direct API calls faster

❌ **(D) Tight Control Needed**

- when you DON'T want model to decide tools

👉 use deterministic backend logic

◇ **8. Key Design Insight (Very Important)**

MCP shifts tool usage from **hardcoded logic → model-driven decision with structured contracts**

◇ **9. Common Mistake (Call this out)**

❌ "MCP is just function calling"

✔ Correct:

MCP is **standardized tool ecosystem**, not just calling functions

---

10. Final Crisp Interview Answer

MCP is a protocol that standardizes how models interact with external tools, similar to how OpenAPI standardizes REST APIs. It enables tool discovery, structured invocation, and decouples the model from backend implementations. I would use it in systems with many dynamic tools, multi-agent setups, or strong backend integration needs. I wouldn't use it for simple or latency-critical workflows where direct API calls are sufficient.

Q4. **Your sub-agent fails silently — no error, just a wrong output. How does your system catch it and what does retry look like?** ***(Tests failure mode thinking — separates good from great)***

1. Start with the principle

I assume LLM outputs are **untrusted by default**. So every sub-agent result must pass **explicit validation gates** before it's accepted.

---

2. How the system *catches* silent failures

You won't catch these with try/catch. You need **post-execution validation layers**:

☑ (A) Schema / Contract Validation (first line of defense)

- Enforce strict structure:
  - JSON schema
  - required fields
  - types & ranges

Example

```
{ "revenue": "5 million" }  ✗ (string instead of number)
```

👉 Rejected immediately.

☑ (B) Deterministic Checks (cheap, high-signal)

- Rules that must always hold:
  - totals match sums
  - dates in valid ranges
  - currency conversions consistent

👉 Catches logical inconsistencies.

☑ (C) Cross-Verification (very important)

- Compare against:
  - retrieved source documents
  - another tool/API
  - earlier steps in state

Example

- Extracted revenue = 500M
- Source doc says 50M → mismatch ❌

☑ (D) Critic / Validator Agent

- Separate agent reviews:
  - correctness
  - completeness
  - hallucination risk

👉 Think: **generator vs reviewer pattern**

☑ (E) Confidence + Evidence Checks

- Require:
  - confidence score
  - citations / spans

👉 Low confidence or missing evidence = reject

☑ (F) Ensemble / Redundancy (for high-stakes)

- Run 2 agents with different prompts/models
- Compare outputs

👉 If disagreement → trigger retry

◇ 3. Where this logic lives

Component	Role
Sub-agent	Generates output
Validator layer	Runs checks
Orchestrator	Decides pass/fail + retry

Important line:
"Sub-agents don't decide correctness—the orchestrator (with validators) does."

◇ 4. What does retry actually look like?

Not just "run again." A good retry is **informed and adaptive**.

🔁 Retry Strategy (step-by-step)

🟣 Step 1: Classify Failure Type

- Schema failure → formatting issue
- Logic mismatch → reasoning issue
- Missing info → retrieval issue

👉 Different failures → different retries

🟣 Step 2: Modify the Next Attempt

Case 1: Schema Failure

- Add stricter instruction:

```
Return ONLY valid JSON. No text.
```

Case 2: Reasoning Failure

- Add:
  - chain-of-thought guidance (hidden)
  - examples (few-shot)
  - constraints

Case 3: Missing Context

- Improve retrieval:
  - more documents
  - better query

🟣 Step 3: Change Something Real

A retry must change at least one of:

- prompt

- context
- model
- temperature

👉 Otherwise, you get the same wrong answer again.

🟣 Step 4: Limit Retries + Escalate

- max retries: 2–3
- if still failing:
  - fallback logic
  - or human-in-the-loop

🔷 5. Example (Interview-Level)

🎯 Use Case: Financial Extraction

Sub-agent output:

```
{ "revenue": 500 }
```

❌ Problem:

- Source doc says **50 million**
- Missing unit → silent failure

☑️ Detection:

1. Cross-check with document → mismatch
2. Validator flags:
   - "unit missing / inconsistent"

🔁 Retry:

Retry 1:

- Add instruction:

```
Always include currency and unit (e.g., USD million)
```

Retry 2 (if needed):

- Provide example:

```
{ "revenue": 50, "unit": "USD million" }
```

☑ **Final Output:**

```
{ "revenue": 50, "unit": "USD million" }
```

◇ 6. Advanced Pattern (Strong Signal in Interview)

🔁 **Self-Refinement Loop**

1. Generate answer
2. Critic reviews
3. Generator improves

👉 Controlled iterative refinement

◇ 7. Common Mistakes (Call these out)

❌ Blind retries
❌ Letting agent self-validate
❌ No observability/logging

✔ Instead:

- structured validation
- centralized decision-making
- traceable state

◇ 8. Final Crisp Answer (Say this if interrupted)

Silent failures are handled through post-execution validation layers like schema checks, deterministic rules, cross-verification with sources, and optional critic agents. The orchestrator evaluates outputs against explicit success criteria. If validation fails, retries are adaptive—modifying prompt, context, or model depending on the failure type, with capped retries and fallback mechanisms.

Q5. **You have 4 agents running in parallel — how do you handle the final aggregation so the output is coherent, not just a dump?** ***(Tests production-level thinking)***

◇ 1. Start with the principle

Parallel agents increase coverage, but **aggregation must impose structure, resolve conflicts, and enforce a single narrative**.

So I design aggregation as a **first-class stage**, not a simple merge.

◇ 2. What goes wrong without proper aggregation

- Outputs are:
  - duplicated
  - conflicting
  - inconsistent in format
  - too verbose

👉 Result = "dump of answers," not usable output

◇ 3. My Aggregation Design (Step-by-Step)

🟣 Step 1: Normalize Outputs (Standard Contract)

All agents must return **structured output**, not free text.

Example schema:

```
{
  "answer": "...",
  "confidence": 0.0-1.0,
  "sources": [],
  "key_points": []
}
```

👉 Without this → aggregation becomes messy

🟣 Step 2: Store in Shared State

Orchestrator collects outputs like:

```
[
  { "agent": "A", "answer": "...", "confidence": 0.8 },
  { "agent": "B", "answer": "...", "confidence": 0.9 }
]
```

👉 Now aggregation has **full visibility**

🟣 Step 3: De-duplication & Alignment

- Merge overlapping points
- Remove repeated info

- Align terminology

👉 Example:

- "Revenue increased"
- "Sales went up"
  → unify into one statement

🟣 Step 4: Conflict Resolution (Very Important)

When agents disagree:

☑ Strategy 1: Confidence-weighted selection

- Higher confidence wins

☑ Strategy 2: Source-based validation

- Prefer answer backed by:
  - citations
  - retrieved documents

☑ Strategy 3: Tie-breaker Agent (Advanced)

- A **resolver/critic agent** decides:
  - which answer is more correct

🟣 Step 5: Synthesis (Not Concatenation)

This is the **core step**.

Instead of joining outputs, I generate a **new unified answer**.

Aggregator Agent does:

- Combine key points
- Create logical flow
- Ensure consistency

👉 Think: "editor," not "collector"

🟣 Step 6: Enforce Output Format

Final output must:

- follow schema
- match user intent
- be concise + structured

🟣 Step 7: Final Validation

Before returning:

- check:
  - completeness
  - no contradictions
  - format correctness

🔷 4. Architecture Pattern

⭐ Parallel → Aggregate → Validate

- Agents (parallel execution)
- Aggregator (synthesis)
- Validator (final check)

🔷 5. Example (Interview-Ready)

🎯 Use Case: Commodity Market Insight

4 agents:

1. Price analysis
2. News analysis
3. Risk detection
4. Trend prediction

❌ Raw Outputs (bad case)

- Agent 1: “Price increased by 5%”
- Agent 2: “Market is bullish”
- Agent 3: “Risk due to supply issues”
- Agent 4: “Upward trend expected”

👉 Just dumping = poor UX

☑️ Aggregated Output (good case)

```
{
  "summary": "Aluminum prices show a 5% increase, driven by positive market
sentiment. However, supply-side risks remain.",
  "trend": "Upward",
  "risks": ["Supply constraints"],
  "confidence": 0.87
}
```

👉 Clean, structured, decision-ready

◇ 6. Advanced Techniques (Strong Signal)

☑ (A) Hierarchical Aggregation

- Group agents:
  - financial → one summary
  - risk → one summary

👉 Then combine

☑ (B) Voting / Ensemble

- Majority agreement
- weighted voting

☑ (C) Role-based Agents

- Each agent owns:
  - specific dimension
    👉 reduces overlap → easier aggregation

◇ 7. Common Mistakes

❌ Just concatenating outputs
❌ No schema enforcement
❌ Ignoring conflicts
❌ No final validation

◇ 8. Key Interview Lines

“Aggregation is a synthesis step, not a merge step.”

“I enforce structured outputs from each agent so aggregation becomes deterministic.”

"Conflicts are resolved using confidence, sources, or a critic agent."

◇ 9. Final Crisp Answer

When multiple agents run in parallel, I first enforce a structured output schema so results are consistent. The orchestrator collects outputs into shared state, then performs deduplication and conflict resolution using confidence scores, source validation, or a critic agent. Instead of concatenating, I use an aggregator step to synthesize a single coherent response, followed by validation to ensure consistency and completeness.